



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 06

NOMBRE COMPLETO: Camacho Ignacio Violeta

N° de Cuenta: 319061345

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 06

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 26-03-2026

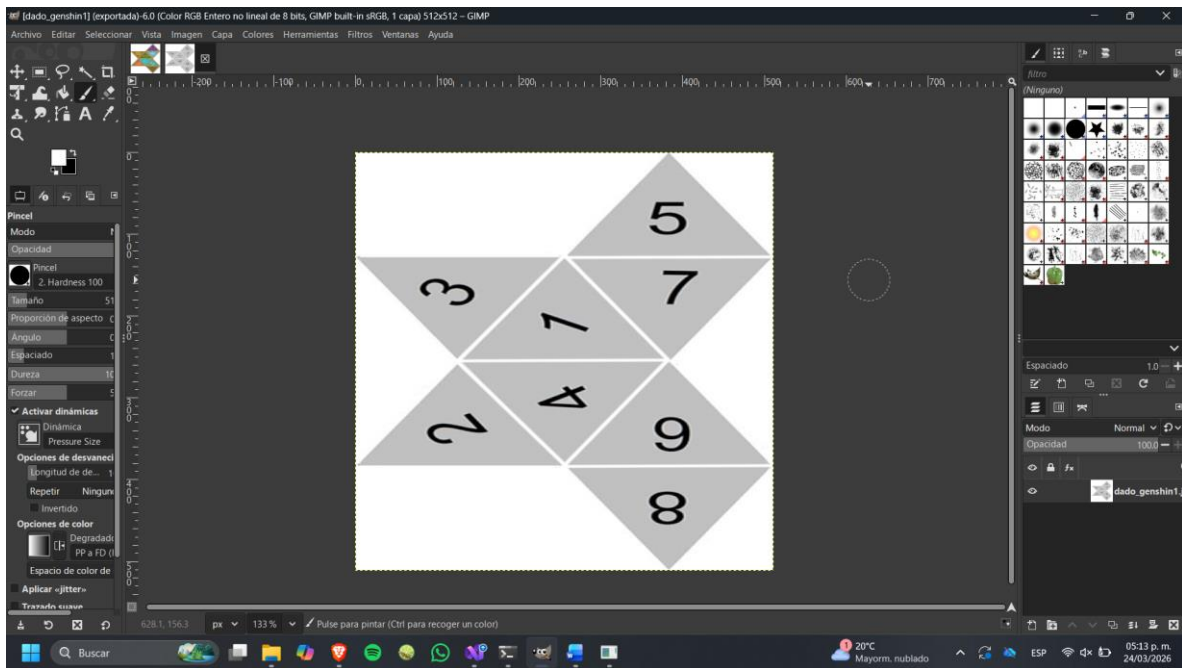
CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

Ejercicio 1: Crear un dado de 8 caras y texturizarlo por medio de código.

Primero se seleccionó la imagen a texturizar y se ajustó dentro de gimp para modificar los números en el sentido que se iban a necesitar siguiendo el sentido de orientación que iban a llevar dentro del dado así como se hizo el re-escalado de la imagen a 512x512px, una vez hechos estos cambios se exportó a .tga



Se define la variable de textura que se va a utilizar y se define la ruta donde se encuentra dicha imagen, para intercambiar entre las dos texturas que se utilizaron solo basta con cambiar el nombre de la imagen de "genshin.tga" a "genshin1.tga"

```
49 Texture genshinTexture;  
50 Texture logoFiTexture;
```

```
351  
352 genshinTexture = Texture("Textures/dado_genshin.tga"); //textura dado-----  
353 genshinTexture.LoadTextureA();
```

El octaedro está formado por 6 vértices principales, 1 superior, 1 inferior y 4 en la parte media, utilizando estos vértices se crean 8 caras triangulares, cada una definida por 3 vértices.

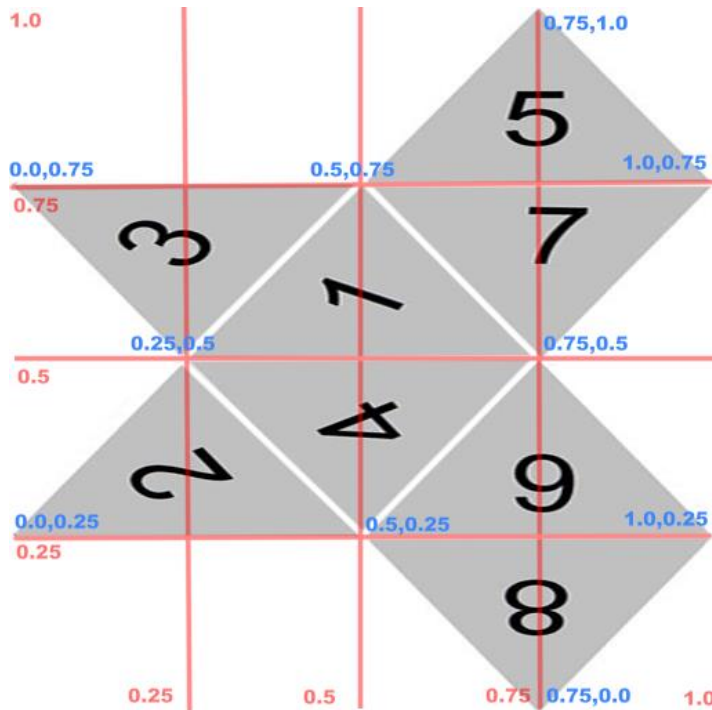
En el código, cada cara se define de forma independiente dentro del arreglo vertices, donde cada vértice contiene (x, y, z, u, v, nx, ny, nz) que equivale a su posición en el espacio 3D, coordenadas de textura y normales para iluminación que en este caso no se utilizaron.

La parte superior cada cara conecta el vértice superior con dos vértices de la base mientras que en la parte inferior cada cara conecta el vértice inferior con dos vértices de la base.

Cada cara del octaedro fue mapeada manualmente, esto se logró ajustando cuidadosamente las coordenadas UV de cada vértice para que coincidan con los triángulos de la textura.

```
259 void CrearOctaedro()
260 {
261     unsigned int indices[] = {
262         0,1,2, 3,4,5, 6,7,8, 9,10,11,
263         12,13,14, 15,16,17, 18,19,20, 21,22,23
264     };
265
266     GLfloat vertices[] = {
267         // ----- SUPERIOR -----
268
269         // cara 1 (arriba) numero 5
270         0.0f,0.7f,0.0f, 0.75f,1.0f, 0.0f,0.0f,0.0f,
271         -0.5f,0.0f,0.5f, 0.5f,0.75f, 0.0f,0.0f,0.0f,
272         0.5f,0.0f,0.5f, 1.0f,0.75f, 0.0f,0.0f,0.0f,
273
274         // cara 2 numero 3
275         0.0f,0.7f,0.0f, 0.75f,0.0f, 0.0f,0.0f,0.0f,
276         0.5f,0.0f,0.5f, 1.0f,0.25f, 0.0f,0.0f,0.0f,
277         0.5f,0.0f,-0.5f, 0.5f,0.25f, 0.0f,0.0f,0.0f,
278
279         // cara 3 numero 2
280         0.0f,0.7f,0.0f, 0.0f,0.25f, 0.0f,0.0f,0.0f,
281         0.5f,0.0f,-0.5f, 0.5f,0.25f, 0.0f,0.0f,0.0f,
282         -0.5f,0.0f,-0.5f, 0.25f,0.5f, 0.0f,0.0f,0.0f,
283
284         // cara 4 numero 8
285         0.0f,0.7f,0.0f, 0.0f,0.75f, 0.0f,0.0f,0.0f,
286         -0.5f,0.0f,-0.5f, 0.25f,0.5f, 0.0f,0.0f,0.0f,
287         -0.5f,0.0f,0.5f, 0.5f,0.75f, 0.0f,0.0f,0.0f,
288
289         // ----- INFERIOR -----
290
291         // cara 5 numero 7
292         0.0f,-0.7f,0.0f, 0.75f,0.5f, 0.0f,0.0f,0.0f,
293         0.5f,0.0f,0.5f, 1.0f,0.75f, 0.0f,0.0f,0.0f,
294         -0.5f,0.0f,0.5f, 0.5f,0.75f, 0.0f,0.0f,0.0f,
295
296         // cara 6 numero 6
297         0.0f,-0.7f,0.0f, 0.75f,0.5f, 0.0f,0.0f,0.0f,
298         0.5f,0.0f,-0.5f, 0.5f,0.25f, 0.0f,0.0f,0.0f,
299         0.5f,0.0f,0.5f, 1.0f,0.25f, 0.0f,0.0f,0.0f,
300
301         // cara 7 numero 4
302         0.0f,-0.7f,0.0f, 0.75f,0.5f, 0.0f,0.0f,0.0f,
303         -0.5f,0.0f,-0.5f, 0.25f,0.5f, 0.0f,0.0f,0.0f,
304         0.5f,0.0f,-0.5f, 0.5f,0.25f, 0.0f,0.0f,0.0f,
305
306         // cara 8 numero 1
307         0.0f,-0.7f,0.0f, 0.75f,0.5f, 0.0f,0.0f,0.0f,
308         -0.5f,0.0f,0.5f, 0.5f,0.75f, 0.0f,0.0f,0.0f,
309         -0.5f,0.0f,-0.5f, 0.25f,0.5f, 0.0f,0.0f,0.0f,
310
311     };
312
313     calcAverageNormals(indices, 24, vertices, 192, 8, 5);
314
315     Mesh* octaedro = new Mesh();
316     octaedro->CreateMesh(vertices, indices, 192, 24);
317     meshList.push_back(octaedro);
318 }
319
320 }
```

Para el cálculo correcto de coordenadas se fueron asignando una por una dependiendo de la orientación que se le quería dar a cada cara del dado apoyándose del vértice inferior o superior que le correspondiera y de la cara que debía tener de qué lado, haciendo uso también de este apa que se creó utilizando como origen la parte inferior izquierda



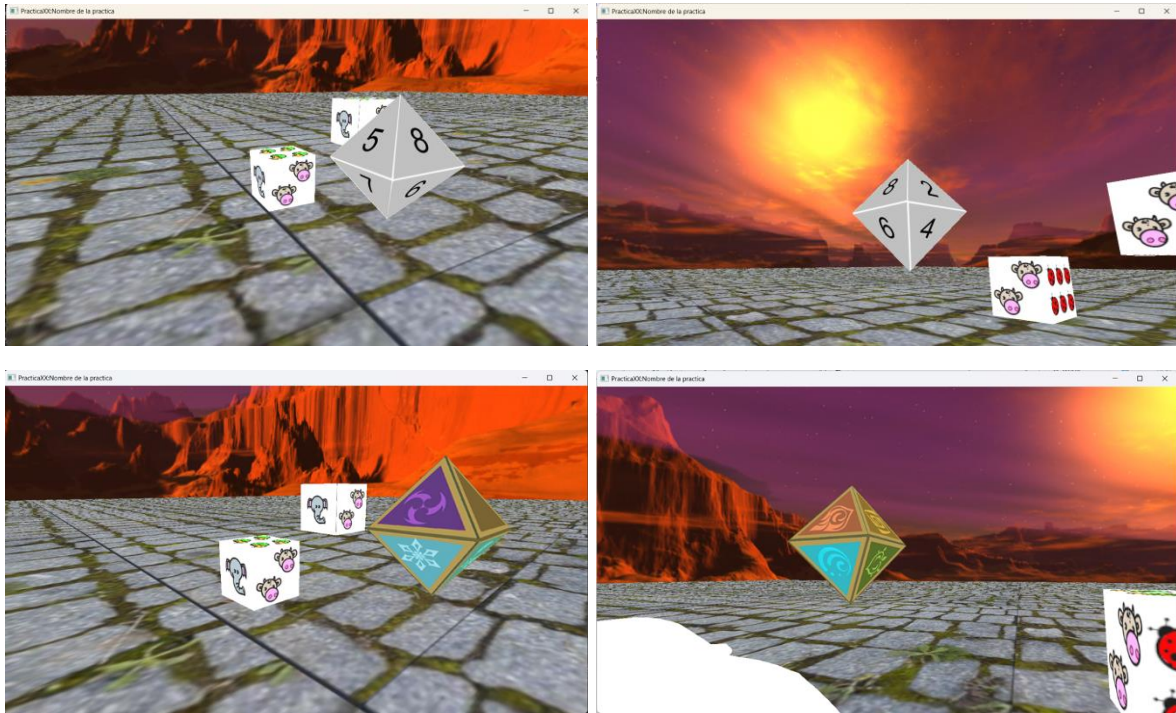
Finalmente se instancia el octaedro y se le asigna la variable de textura que contiene la imagen del dado correspondiente

```

418 //-----
419
420 model = glm::mat4(1.0);
421 model = glm::translate(model, glm::vec3(3.0f, 5.0f, 2.0f)); // posición diferente
422 model = glm::scale(model, glm::vec3(2.0f, 2.0f, 2.0f));
423
424 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
425
426 genshinTexture.UseTexture(); // textura
427 meshList[5]->RenderMesh(); // el octaedro
428
429 //-----
430

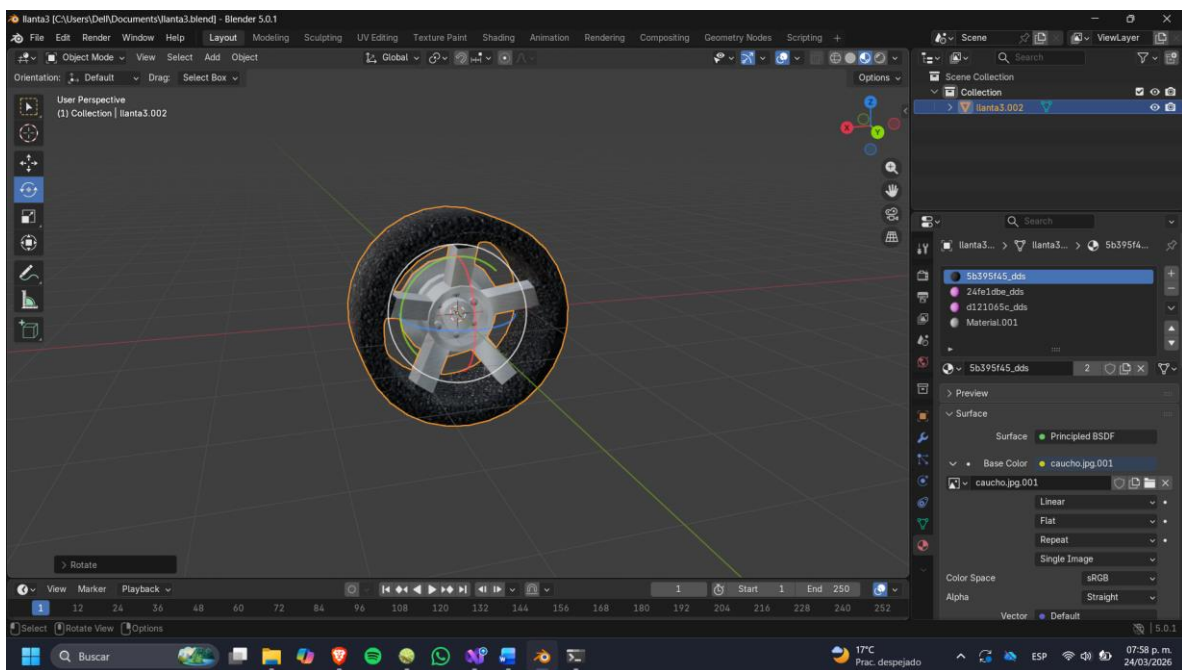
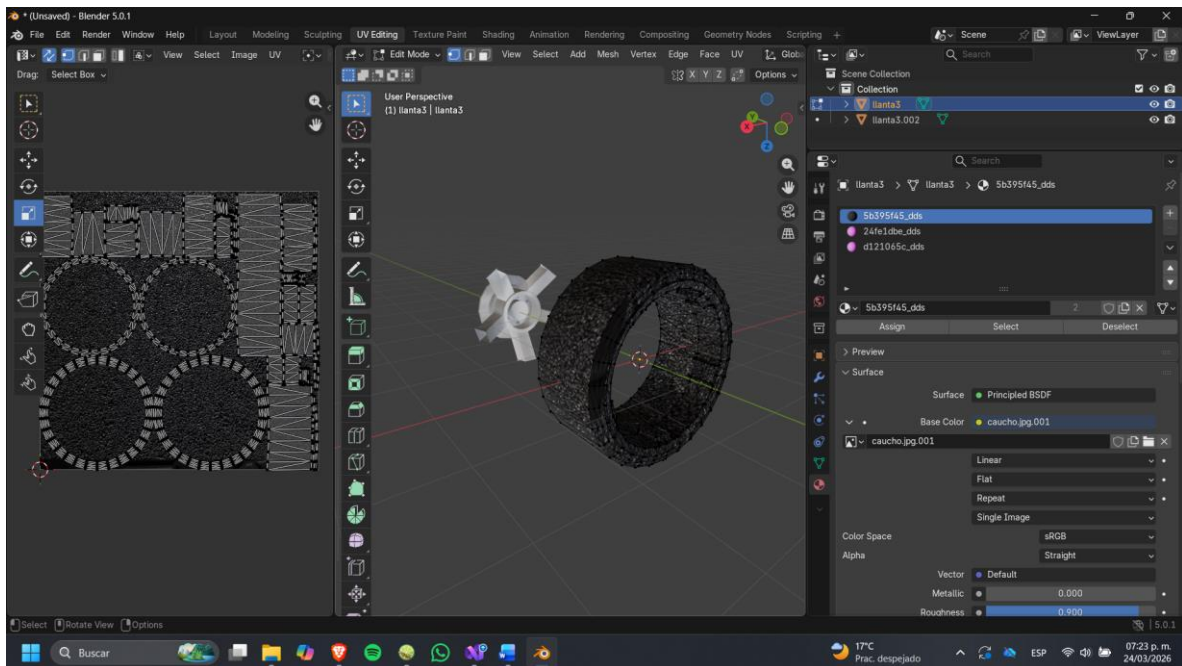
```

Ejecución de ambas texturas



Ejercicio 2: Importar el modelo de su coche con sus 4 llantas acomodadas y tener texturizadas las 4 llantas (diferenciar caucho y rin)

En blender se importó el modelo de la llanta para separar las caras del rin y de la parte de caucho, seleccionando una a la vez y utilizando el UV mapping se le aplicó una textura de metal al rin y una de caucho a la llanta, posteriormente antes de exportar se realizó una rotación de esta misma llanta dentro de blender para crear las demás llantas y evitar hacer este proceso más tarde en el código



Dentro del código se crearon 4 variables para cada una de las 4 llantas ya que cada una tiene una posición y tecla distinta para moverse sin embargo debido a su orientación se utilizó el mismo modelo para las llantas delantera y trasera de la derecha y el segundo modelo para las delantera y trasera de la izquierda

```

363 //acá llamar a los modelos de los ejercicios-----
364 Carro_cuerpo = Model();
365 Carro_cuerpo.LoadModel("Models/cuerpo.obj");
366
367 Carro_cofre = Model();
368 Carro_cofre.LoadModel("Models/cofre.obj");
369
370 Carro_llanta1 = Model();
371 Carro_llanta1.LoadModel("Models/llanta1.obj");
372
373 Carro_llanta2 = Model();
374 Carro_llanta2.LoadModel("Models/llanta1.obj");
375
376 Carro_llanta3 = Model();
377 Carro_llanta3.LoadModel("Models/llanta3.obj");
378
379 Carro_llanta4 = Model();
380 Carro_llanta4.LoadModel("Models/llanta3.obj");

```



Finalmente se colocó el código correspondiente de la práctica pasada tanto en el archivo main como en Window.cpp y window.h para conservar la movilidad del auto con las teclas n y m, así como el movimiento de las 4 llantas para adelante y hacia atrás con las teclas 1 y 2, y el abrir y cerrar el cofre con 4 y 5. Se crearon las instancias del cuerpo sin llantas, el cofre y las 4 llantas.

```
E06-319061345.cpp Window.cpp Window.h
Project6c (Ámbito global)
484 //-----
485
486 //----- Ejercicio 2 practica -----
487
488 // ===== cuerpo carro =====
489
490
491
492 model = glm::mat4(1.0);
493 model = glm::translate(model, glm::vec3(mainWindow.getmuevex(), -1.0f, -1.5f)); //mover padre mover todo
494
495 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
496 dadoTexture.UseTexture();
497 Carro_cuerpo.RenderModel(); //modelo sin las 4 llantas ni cofre
498
499 modelaux = model;
500
501
502 // ===== cofre =====
503
504 model = modelaux;
505 model = glm::translate(model, glm::vec3(2.0f, 3.0f, 0.0f));
506
507 model = glm::rotate(
508     model,
509     mainWindow.getarticulacion5() * toRadians,
510     glm::vec3(0.0f, 0.0f, 1.0f) // eje Z
511 );
512
513
514 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
515 Carro_cofre.RenderModel();
516
517
518 // ===== llanta 1 delantera izq =====
519
520 model = modelaux;
521 model = glm::translate(model, glm::vec3(4.5f, 1.0f, -3.0f));
522
523 model = glm::rotate(
524     model
```

```
E06-319061345.cpp Window.cpp Window.h
Project6c (Ámbito global)
535
536 model = glm::rotate(
537     model,
538     mainWindow.getarticulacion2() * toRadians,
539     glm::vec3(0.0f, 0.0f, 1.0f) // eje Z
540 );
541 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
542 Carro_llanta2.RenderModel();
543
544 // ===== llanta3 delantera derecha =====
545
546 model = modelaux;
547 model = glm::translate(model, glm::vec3(4.5f, 1.0f, 3.0f));
548
549 model = glm::rotate(
550     model,
551     mainWindow.getarticulacion3() * toRadians,
552     glm::vec3(0.0f, 0.0f, 1.0f) // eje Z
553 );
554 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
555 Carro_llanta3.RenderModel();
556
557 // ===== llanta4 trasera derecha =====
558
559 model = modelaux;
560 model = glm::translate(model, glm::vec3(-4.5f, 1.0f, 3.0f));
561
562 model = glm::rotate(
563     model,
564     mainWindow.getarticulacion4() * toRadians,
565     glm::vec3(0.0f, 0.0f, 1.0f) // eje Z
566 );
567
568 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
569 Carro_llanta4.RenderModel();
```

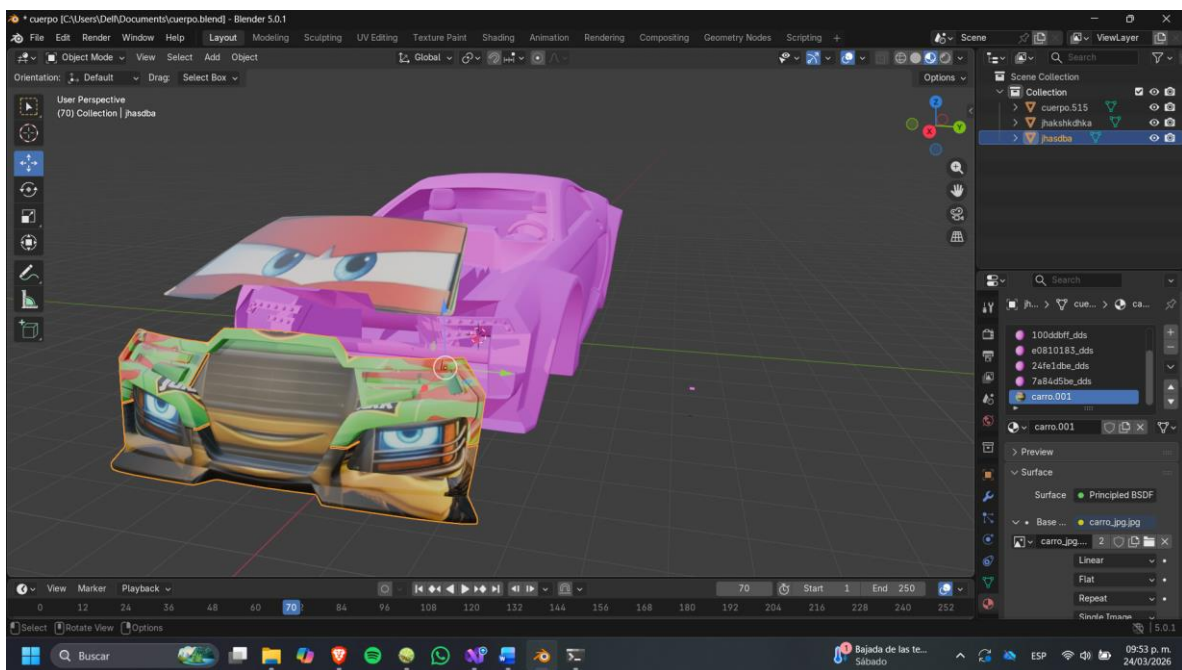
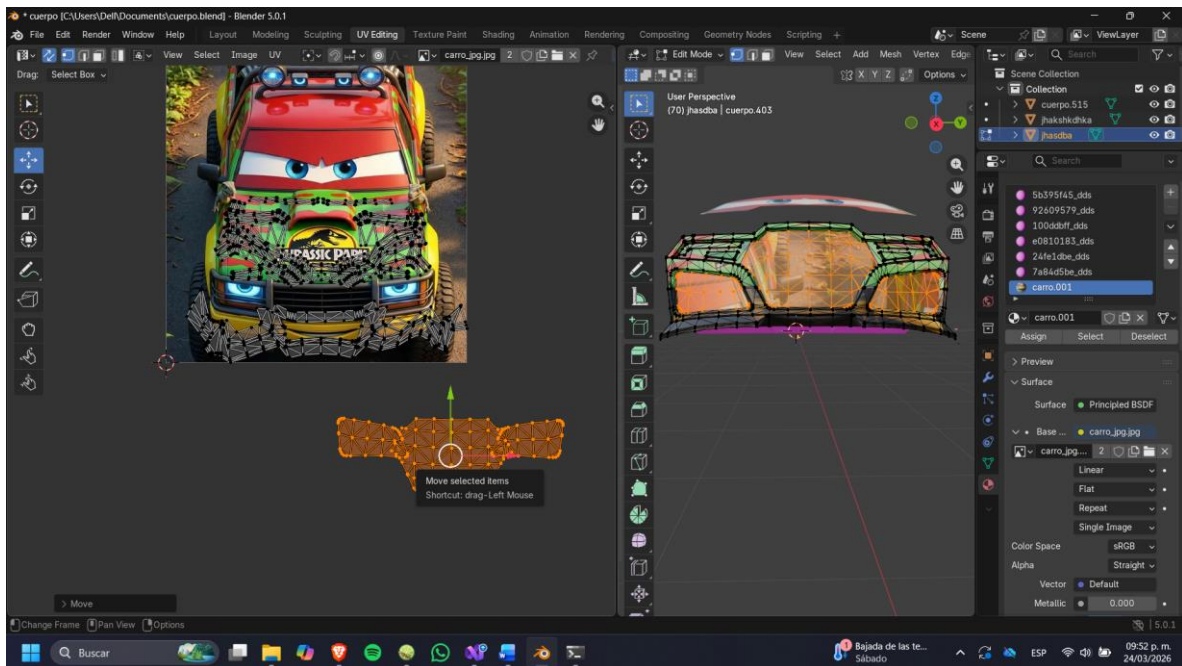

Ejercicio 3: Texturizar la cara del personaje de la imagen tipo cars en el espejo (ojos) y detalles en cofre y parrilla de su propio modelo de coche

Primero se comenzó con el cofre siguiendo el mismo proceso que con la llanta, se importó el .obj de la parte del cofre ya separada y se aplicó la textura de imagen, con UV mapping se ajustó la posición que debía desplegar y se exportó nuevamente como .obj



Se siguió el mismo proceso con el cuerpo que con la llanta al separar primero las partes a texturizar y con UV mapping ajustar cada una





Al exportar el modelo e importarlo dentro del código es posible ver las partes del modelo que tienen textura mientras que las que no la tienen se multiplican por el color blanco que se le asignó, esto se conservó de este modo al principio para visualizar mejor que la textura mapeada sobre el cobre y cara estuviese cargada correctamente





Por propósitos estéticos al final se le añadió una textura metálica en color verde



2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla

En un punto al momento de exportar el modelo en blender o de importarlo dentro de visual studio la textura no se mostraba correctamente, se logró identificar que el problema era que la imagen ajustada para la textura se había guardado en una carpeta diferente, se arregló colocando la imagen correcta en la carpeta de texturas correspondiente



3.- Conclusión:

a. *Los ejercicios del reporte: Complejidad, Explicación.*

Los ejercicios realizados en esta práctica presentaron un nivel de complejidad medio-alto, principalmente debido al manejo de texturas en modelos 3D y la correcta asignación de coordenadas UV.

En el primer ejercicio, la dificultad radicó en mapear manualmente cada cara del octaedro, lo cual requirió precisión en la asignación de coordenadas para evitar errores de orientación en la textura.

En los ejercicios posteriores, especialmente al trabajar con modelos importados desde Blender, la complejidad aumentó al tener que separar materiales, aplicar texturas diferentes y asegurar que estas se visualizaran correctamente en el entorno de programación.

La explicación de los ejercicios permitió comprender mejor la relación entre el modelado en herramientas externas y su implementación en código, reforzando conceptos importantes como el mapeo UV, la estructuración de vértices y la manipulación de objetos 3D.

b. *Comentarios generales: Faltó explicar a detalle, ir más lento en alguna explicación, otros comentarios y sugerencias para mejorar desarrollo de la práctica*

algunos procedimientos, especialmente los relacionados con la exportación de modelos y la correcta aplicación de texturas, podrían explicarse con mayor detalle. En particular, sería útil que se profundizara más en el flujo completo entre Blender y el entorno de programación, ya que pequeños errores en la exportación o configuración pueden provocar fallos difíciles de identificar

c. Conclusión

esta práctica permitió desarrollar habilidades importantes en el manejo de gráficos por computadora, específicamente en la aplicación de texturas sobre modelos 3D tanto generados por código como importados desde herramientas externas, se logró comprender la importancia del mapeo UV y la correcta organización de los modelos para su posterior uso en un entorno de programación.

A pesar de los problemas presentados en un momento con la visualización de texturas, estos forman parte del proceso de aprendizaje y contribuyen a una mejor comprensión de la interacción entre software de modelado y programación gráfica, gracias a que se lograron resolver este tipo de problemas es que en general la práctica cumplió su objetivo al reforzar conocimientos clave sobre texturizado y fomentar la resolución de problemas en escenarios reales.

4.- Bibliografía en formato APA

- Blender Foundation. (2024). Blender Manual. Recuperado de <https://docs.blender.org/manual/en/latest/>
- OpenGL. (2023). OpenGL Documentation. Recuperado de <https://www.opengl.org/documentation/>
- LearnOpenGL. (2023). Textures. Recuperado de <https://learnopengl.com/Getting-started/Textures>
- GIMP Development Team. (2024). GIMP Documentation. Recuperado de <https://docs.gimp.org/>
- Wikipedia. (2024). Mapeo UV. Recuperado de https://es.wikipedia.org/wiki/Mapeo_UV